

Leveraging RISC-V for HW/SW Codesign of Flexible and Efficient TinyML SoCs

Angelo Garofalo

University of Bologna
40126 Bologna, Italy

Luca Benini

ETH Zürich
8092 Zürich, Switzerland

Editor's notes:

This keynote article presents a comprehensive and impactful contribution to energy-efficient TinyML through the design of domain-specific RISC-V instruction set architecture (ISA) extensions for Edge AI workloads. The authors provide detailed hardware–software codesign contributions, including sub-byte SIMD arithmetic, fused mac-load operations, and vector lockstep execution within tightly coupled compute clusters. This keynote exemplifies the potential of open ISAs and specialized microarchitectures to advance the next generation of TinyML and Edge AI systems.

—*Theocharis Theocharides, University of Cyprus, Cyprus*

—*Marian Verhelst, KU Leuven, Belgium*

—*Vijay Janapa Reddy, Harvard University, USA*

—*Evgeni Gousev, Qualcomm, USA*

significantly reduces the communication energy that usually dominates the power budget of extreme edge devices.

At the same time, TinyML entails major challenges in the design of low-power AI-capable digital hardware or IoT end nodes, which must cope with a heterogeneity of workloads: first, they need to be versatile and software-program-

■ **RECENT YEARS HAVE** seen a growing interest in deploying artificial intelligence (AI) in extreme-edge Internet-of-Things (IoT) applications, including autonomous nanodrones, wearable healthcare, and augmented reality. This wide spectrum of application domains has been denoted as TinyML [1].

TinyML ultimately aims at embedding AI as close as possible to sensors and actuators, reducing privacy concerns since the raw private data acquired by sensors can be distilled locally through AI algorithms, and only high-level features are to be sent to the cloud for data aggregation. This approach also

able to perform tasks such as data acquisition from sensors, preprocessing chains, control tasks, and traditional digital signal processing (DSP) kernels; second, they must feature sufficient computational capabilities to run more complex workloads, including AI, within the real-time constraints that the applications impose. All of this must be in a compact form factor to ensure low cost, wearability, and embeddability in IoT end nodes, all while maintaining a very tight power budget (tens of milliwatts) to guarantee long-term battery-powered operation.

A key characteristic of deep neural networks (DNNs) is their suitability to low-bitwidth arithmetic quantization, where weights and activations are represented in low-precision integer formats, often in mixed precisions (i.e., activations represented

Digital Object Identifier 10.1109/MDAT.2025.3573686

Date of publication: 26 May 2025; date of current version: 29 August 2025.

with 8 b and weights with 4-b integers). Posttraining quantization [2] and quantization-aware training [3] are widely adopted to minimize the accuracy drop implied by aggressive quantization [4].

On the hardware side, beyond memory efficiency, low-bitwidth representations provide an opportunity to lower computation costs since low-bitwidth integer arithmetic is inherently more efficient than floating-point computation. Custom DNN accelerators take full advantage of mixed-precision integer arithmetic, targeting data formats down to 2 bits for extreme energy efficiency [5], [6], [7]. However, accelerators have drawbacks that make them a high-risk design choice for ultralow-power IoT end nodes: high fabrication costs per unit, limited flexibility, and limited software support. These factors jointly make it very challenging for accelerators to keep pace with rapidly evolving AI models.

A cost-effective and efficient solution emerges from the specialization of instruction processors that are ubiquitous in the dominant computing platform for IoT end nodes, namely, the microcontroller unit (MCU). Significant efforts in both industry and academia focus on enabling quantized neural network (QNN) execution on general-purpose (GP) cores for MCUs, targeting byte, subbyte, and mixed-precision integer QNN kernels. ARM has matured 16- and 8-bit QNN software libraries for standard Cortex-M processors. These libraries have been extended (see [8]) to target subbyte and mixed-precision formats. However, classical Cortex-M processors lack instruction set architecture (ISA) support for low-bitwidth integer single-instruction-multiple-data (SIMD) arithmetic operations: the ARMv7E-M ISA supports only a dual 16-bit SIMD MAC operation, while the latest ARMv8.1-M [9] provides support down to 8-bit SIMD but not lower. This results in inefficiencies in computations with lower bitwidths: manipulating data at bitwidths lower than those natively supported in hardware requires additional instructions for bit manipulation and casting.

The RISC-V ISA is in a similar situation: although open-source software libraries [10] have been proposed to optimize QNN execution on parallel compute clusters of RISC-V processors, targeting 32-bit down to 2-bit and binary QNN kernels, the same limitation persists at the ISA level. The RISC-V ratified packed SIMD instructions (P) only support operations down to 8-bit vectors. Consequently, while subbyte mixed-precision quantization methods

continue to effectively reduce the DNN model's footprint on ARM and RISC-V processors, they also introduce significant performance and energy overheads during computation.

However, a key feature of the RISC-V ISA is its extensibility with custom instructions. This creates a viable path to specialize instruction processors for TinyML at low platform cost, without reducing performance and energy efficiency on other GP compute and control tasks. In this work, we show how the joint efficiency and flexibility challenge can be addressed by architectural and microarchitectural codesign techniques that exploit the RISC-V ISA and the parallel ultralow-power (PULP) principles [11].

- We detail our experience in introducing different sets of domain-specific RISC-V ISA extensions for TinyML, targeting byte and subbyte symmetric SIMD integer arithmetic, a fused mac-load instruction to improve hardware utilization on compute-intense kernels, and status-based executions of SIMD mixed-precision operations. These extensions are implemented on a baseline RV32 MCU core, RI5CY [12], featuring a 32-bit, four-stage, in-order, single-issue pipeline.
- To exploit the parallelism of QNN workloads, we review how we integrated the RISC-V cores in multiple-instruction-multiple-data (MIMD) compute clusters, showing a near-ideal speed-up compared to the single-core execution; we focus on how we extended the cluster to operate in vector lockstep execution mode (VLEM) during the execution of QNN convolution innermost loop (matrix multiplication), with only the *leader* core handling the instruction fetching and dispatching and the other cores that act as processing elements. This solution allows us to reduce the power consumption by 35% compared to the baseline MIMD execution and achieve 1.6× energy efficiency improvement.
- We report on an implementation of the RISC-V cores and the compute clusters in GlobalFoundries 22-nm FDX technology to evaluate the area, power, and performance overhead of the proposed solutions at the core and cluster level against the baselines. Results show a limited area overhead of 5.7%, with a negligible power overhead at the cluster level.
- We benchmark our solutions on QNN convolution layers, demonstrating up to 20× better

performance and 27× better energy efficiency compared to the baselines implemented in the same technology. Furthermore, we compare our solutions against state-of-the-art architectures and software, showing up to two orders of magnitude better performance than the commercially available ARM Cortex-M cores leveraging the ARMv7E-M ISA and the Cortex-M55 implementing the newest ARMv8.1-M ISA. Moreover, we demonstrate one order of magnitude better energy efficiency than a 22-nm FDX implementation of an ARM Cortex-M4.

The RISC-V PULP approach demonstrates that it is possible to achieve ASIC-level efficiency for QNN model inference on MCU-class devices, utilizing software programmability. By integrating advanced architectural strategies, power-aware microarchitectural optimizations, and cutting-edge near-threshold FDX technology, we attain performance improvements in more than one order of magnitude beyond current MCUs.

Related work

Multiprecision low-bitwidth arithmetic is a well-established approach to deploy memory and power-hungry AI models at the extreme edge, offering two advantages: it lowers the memory footprint and may reduce latency and energy of the computation if the underlying hardware natively supports low-bitwidth operations. This motivated the exploitation of low-precision parallel integer hardware datapaths. This trend, reviewed in this section, impacts three main computing platforms: dedicated accelerators, FPGAs, and MCUs. We leave out GP-GPUs, which are ubiquitous in high-performance systems but are quite rare in IoT end nodes.

Dedicated accelerators

Dedicated accelerators lead in performance and energy efficiency for QNN workloads [13]. With highly specialized datapaths, they deliver 1–100 Gops/s and efficiencies of 10–100 Tops/W. For instance, Orlando [14] achieves several Tops/W of efficiency.

The effectiveness of low-bitwidth arithmetic architectures for QNN workloads has been well demonstrated in dedicated accelerators [15]. For instance, [6] introduces a bit-serial MAC unit supporting 1- to 16-bit multiprecision operands, with the second

operand fixed at a single bit. Optimized for efficiency, the system achieves 50.6 Tops/W at 184-Gops/s throughput. Similarly, [5] presents a DNN accelerator featuring an energy-scalable, multiprecision integer unit, delivering 76 Gops/s at up to 10 Tops/W. This design employs a parallel multiply-and-add architecture using Booth–Wallace multipliers, reconfigurable for 4-to-16 × 16 b operations.

Despite their performance, these accelerators lack flexibility, complicating the end-to-end deployment of full-scale DNNs. Although modern architectures offer some degree of reconfigurability (e.g., supporting various convolution kernel sizes or handling inception layers), they usually lack the flexibility to support a wide range of DNN models.

In IoT, flexibility and mature software support is essential. DNN inference is often part of larger applications that require peripheral handling, linear algebra, domain-to-domain transforms (sometimes involving floating-point numbers), and wireless transmission of compressed results. Accelerators need to be coupled with a GP processor core to ensure flexibility, but the limited spectrum of use cases covered by the accelerator makes it hard to justify its silicon budget. The limited adaptability and high cost per device hinder the widespread diffusion of dedicated accelerators on extreme-edge IoT sensor nodes: even though ARM announced the availability of the ETHOS-U family of accelerators [called neural processing units (NPU)] for MPUs at the beginning of 2020, the first commercial chips featuring ETHOS-U NPUs are only sampling at the end of 2024.

FPGAs

The development of heterogeneous FPGAs, such as the Xilinx Zynq family, has enhanced flexibility for CNN acceleration. By integrating GP processors, these FPGAs simplify program flow management, I/O handling, memory access, and communication, making them easier to program and interface with external devices and sensors.

FPGAs often include DSP-capable hardware and operate within a Watt-scale power envelope. To improve performance and energy efficiency, it is critical to reduce numerical precision. Although many solutions use 16-bit fixed-point operands [16], there is growing interest in byte and subbyte arithmetic. For instance, Qiu et al. [17] introduced an accelerator for 8- and 4-bit data on a Xilinx

Zynq, and others [18] explored ternary and binary networks.

Unfortunately, most FPGA solutions exceed the power constraints of the IoT end nodes. As an exception, Lattice's Sense-AI family [19] offers ultra-low-power FPGAs tailored for always-on AI within a 1-mW to 1-W power budget. However, their limited LUT capacity and relatively higher cost make them less widespread than traditional MCUs in cost-sensitive applications. Furthermore, while FPGAs offer extreme flexibility, their programming is still quite complex, thereby hindering widespread adoption by IoT developers, who prioritize the flexibility and streamlined software support of MCUs.

Software programmable solutions

Commercially available software-programmable GP processors provide the highest flexibility for the deployment of QNNs at the extreme edge. However, single-core ultralow-power MCUs are not fast enough to sustain QNN workloads. High-end embedded platforms, such as Raspberry Pi [20], boost performance with fast, multicore 64-bit processors. Another widespread architectural template couples a programmable core with a dedicated accelerator. Notable examples are Kendryte [21], a dual-core RISC-V SoC outfitted with a CNN accelerator for AI applications, or Edge TPU¹ coupled with a CPU. Although these heterogeneous platforms are flexible and efficient, their power consumption is high, and their cost is too high for the extreme edge. In many IoT applications, cost constraints are very tight, and it is desirable to reduce power and cost by having an MCU with boosted computing capabilities.

To enhance the performance of MCU systems, a common approach is to extend cores by enhancing their ISAs with custom instructions tailored for specific application domains. ARMv7e provides SIMD instructions for 16-bit data, and the current generation of Cortex-M cores supports this instruction set. Commercial embodiments of this ISA show a power envelope of few milliwatts, fitting the power budget of the IoT end nodes. For example, STMicroelectronics proposed low-end (STM32L4² family of microcontrollers based on the Cortex-M4 cores) and high-end (STM32H7³ family embedding the Cortex-M7 cores)

microcontrollers supporting DL processing at the edge. While the newest ARMv8-M ISA [9] supports 8-bit data with SIMD operations, eventually with an optional 64-b wide vector extension called Helium, smaller SIMD formats are not supported. However, evaluation boards integrating the ARM Cortex-M55 [22] with the Helium ISA are just becoming commercially available.

On the RISC-V side, the XpulpV2 ISA extensions [12] are designed for efficient DSP, supporting SIMD down to 8-bit vector data. Leveraging this ISA, near-threshold multicore heterogeneous platforms have been built to push the performance and efficiency of QNN workloads. For instance, the commercially available GAP-8 [23] embeds a cluster of eight RISC-V cores and a CNN-specialized accelerator that can give the MCU a 5× to 10× boost in energy efficiency.

To provide these architectures with an efficient software back end, the literature presents several solutions that achieve promising results on QNN workloads. It is worth citing the CMSIS-NN [24] library, developed by ARM to target their Cortex-M cores. As an additional contribution, this library has been extended to support heavily quantized and mixed-precision kernels [25]. On the RISC-V side, PULP-NN [10] provides a solid back end for RISC-V-based multiprocessor systems, supporting byte as well as subbyte and mixed-precision QNN kernels.

Unfortunately, even if subbyte integer arithmetic is already adopted in training and quantization flows and ASIC/FPGA-based systems, it is not natively supported in the new generation of architectural solutions for MCU-based systems. The new generation of the ARM ISA for Cortex-M core [22], tailored for the QNN workload, features hardware loops, conditional execution instructions, and 8-bit SIMD instructions like those presented in [12]. However, it will not support low-precision SIMD arithmetic.

In this article, we bridge this ISA and hardware gap to improve efficiency on heavily quantized NN workloads at the extreme edge of the IoT on fully programmable MCU devices. In the next sections, we move from the work presented in [26] and describe an energy-efficient multiprecision integer dot-product (dotp) unit, supporting SIMD vector operations on 16- down to 2-bit precision elements. This unit is integrated into the RISC-V cores of a PULP cluster. The focus of the following sections is on demonstrating the combined effect of ISA extensions

¹<https://cloud.google.com/edge-tpu>

²<https://www.st.com/resource/en/datasheet/stm32l476je.pdf>

³<https://www.st.com/resource/en/datasheet/stm32h743bi.pdf>

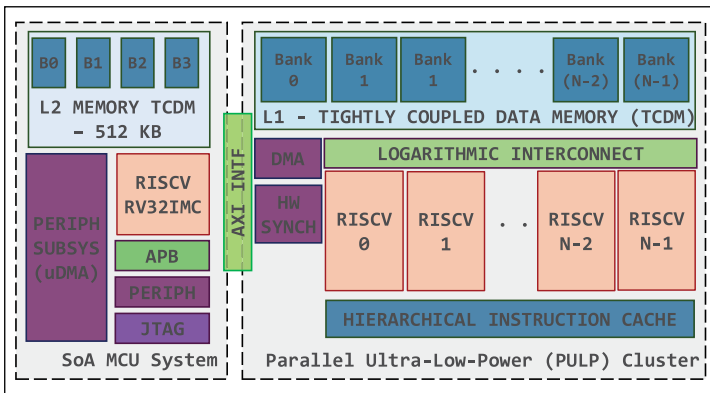


Figure 1. Architecture of the PULP platform, consisting of an MCU system accelerated by a parallel cluster of N configurable RISC-V cores.

and microarchitectural optimizations at the core and cluster level to achieve an order-of-magnitude improvement in MCU performance and efficiency, without compromising flexibility.

Background

PULP cluster

The PULP platform is an open-source computing architecture that aims to improve energy efficiency through near-threshold computing, leveraging parallelism to counterbalance performance reductions that occur at reduced voltage levels [11]. Our baseline platform focuses on the PULP cluster, depicted in Figure 1, which encompasses eight RISC-V cores [12], each of which is built with a four-stage, in-order, single-issue pipeline and runs on the RISC-V RV32IMC-XpulpV2 ISA. This XpulpV2 is a custom extension of the RISC-V ISA [12], specifically tailored for efficient DSP functions. It incorporates hardware loops, post-modified load/store instructions, and SIMD operations down to 8-bit integer vector operands.

The cores in the cluster share data on a tightly coupled data memory (TCDM), organized into several banks with a banking factor of two. We instantiate a cluster with 128-kb TCDM and eight cores, resulting in a total of 16 banks. The TCDM is connected to the cores via a low-latency logarithmic interconnect, enabling each core to access it in two cycles, when there are no banking conflicts.

PULP cluster is added to a traditional single-core MCU using an AXI interface and includes a direct memory access (DMA) unit. This DMA facilitates the

movement of data between the TCDM and the secondary memory of the microcontroller, which also accommodates the program instructions required by the cluster. On the instruction fetch ports, each core retrieves its instructions from a two-tiered instruction cache architecture, where the first level cache is dedicated to individual cores and the second level is shared.

In addition, the cluster features a hardware synchronization unit designed to enable efficient synchronization and thread management. This capability allows for precise parallelism with reduced overhead, optimizing energy efficiency by placing any core that is idle while waiting for a barrier into a completely clock-gated condition.

QNN execution model

The software stack for the cluster relies on the open-source PULP-NN library.⁴ This library includes a diverse range of kernels designed for the fast execution of QNNs on PULP platforms utilizing the RISC-V ISA. The kernels within the library are specifically crafted to leverage the cluster-level parallelism of PULP and the custom instruction extensions provided by the processors' cores, including the Xpulp instructions introduced above.

For efficient inference of deep convolutional QNNs, PULPNN employs a height-width-channel (HWC) data arrangement specifically designed for microcontrollers, organizing the convolution operation into three primary stages:

im2col: At this phase, the 3-D input activations are processed in the (H, W, C) configuration, which organizes the comprehensive receptive field for a specific output location into a 1-D vector that spans the filter and input channel dimensions. This transformation allows the convolutional layer to be executed through a scalar multiplication involving this vector and the flattened weights. PULP-NN carries out this process for two output pixels simultaneously, generating two separate *im2col buffers*, with each comprising $C_{in} \times F \times F$ elements.

Matrix multiplication: This stage constitutes the heart of the convolution process. It executes a summation of the dot products between the present *im2col buffer* and the sets of filters, producing higher precision results in 32-bit intermediate output activations.

⁴<https://github.com/pulp-platform/pulp-nn>

To improve the computing efficiency during this phase, the kernel utilizes the core's underlying ISA, incorporating specialized extensions. These extensions include hardware loops (*lp.setup*), load operations with postincrement (*p.lw*), and SIMD sum-of-dot-product (*sdotp*) instructions, which achieve four multiply-accumulate (MAC) operations with a latency of just one cycle when working with 8-bit SIMD operands. Furthermore, there is significant data reuse at the register-file (RF) level.

In this context, the innermost loop of the *MatMul* phase handles activations sourced from two *im2col* buffers, which correspond to two spatially neighboring output pixels, along with quantized weights derived from four filter banks associated with four output channels. Leveraging data locality within the RF, the computation of eight output pixels is accomplished for each iteration of the *MatMul* inner loop, which involves four channels $\times$ two adjacent pixels.

This 4×2 structure results from a design space exploration to find the data reuse condition that maximizes throughput [10]. Given the limited number of registers available on edge RISC-V core (32 registers) register file for storing operands and accumulators, no additional reuse can be exploited: wider *MatMul* structures (e.g., 4×4) would cause operand spilling due to limited registers. This would result in the compiler frequently transferring operands to and from the stack, leading to significant overhead [10].

Quantization: To obtain low-bitwidth representations of accumulation results, 32-bit accumulators are normalized and quantized. This process entails a MAC operation, a shift, and a clipping instruction.

RISC-V ISA extensions for TinyML

This section reviews the design of custom RISC-V instructions tailored to accelerate subbyte and mixed-precision TinyML applications. We first illustrate the instructions' encoding and their working principles. Next, we discuss the performance improvements that these instructions provide when executing QNN kernels, along with the microarchitectural costs of implementing them in a processor core for MCUs.

xpulpnn

The *xpulpnn* extension accelerates SIMD operations such as addition, subtraction, multiplication, *dotp*, and *sdotp* on packed vectors of 8-bit (*byte*), 4-bit (*nibble*), and 2-bit (*crumb*) elements, as summarized in Figure 2a. The input registers support

signed, unsigned, or mixed formats, while the output register, used as the accumulator or scalar in *dotp* and *sdotp*, can also be signed or unsigned. Additional instructions, including SIMD *maximum*, *minimum*, *average*, and shift operations, enhance nonlinear TinyML functions such as ReLU, pooling, batch normalization, and softmax.

By leveraging *xpulpnn* instructions inferred in C code through built-in functions (as discussed in “Programming model”) and optimizing data reuse at the RF level, computational efficiency in subbyte PULP-NN *MatMul* kernels improves significantly. For 4×2 structured kernels (see “QNNs execution model”) using uniform subbyte precision of the operands, the computation efficiency (CEF)—the ratio of SIMD *sdotp* operations to total executed instructions (including loads, stores, loop control, and data manipulation beyond *sdotp*)—reaches 0.57 in the innermost loop. In nonoptimized *MatMuls* running on in-order single-issue load-store architectures, this value is typically 0.33, and it is anyway limited to 1 by construction (i.e., the number of SIMD *Dopt-Units* in the processor's datapath). Figure 3b illustrates the pseudoassembly for PULP-NN's *MatMul* inner loop, showing all utilized RF resources.

Two key factors limit higher CEF: 1) the presence of load operations within kernel loops and 2) constraints of RF resources, comprising 32-bit registers in mid-range RISC-V microcontrollers (RV32 ISA). These limitations restrict the number of weights, activations, addresses, and accumulators that can be managed without frequent operand spills onto the stack, which drastically reduces performance.

Mac-load (m&l) instruction

To address limitations and improve CEF in PULP-NN *MatMuls*, we proposed the *mac&load* (*m&l*) instruction, leveraging the deterministic memory access patterns of dense matrix multiplications. In the innermost loop, the memory addresses for operands needed in the $(i + 1)$ th iteration can be determined during the i th iteration, enabling overlap of memory access (for the next iteration's data) with computation (for the current iteration).

The *m&l* instruction fuses SIMD *dotp*/*sdotp* operations with *load* in a single instruction. This is feasible since: 1) the microarchitectural datapaths for *dotp* and *load* are independent and can operate in parallel and 2) memory address increments are constant (regular accesses) and hardwired into the

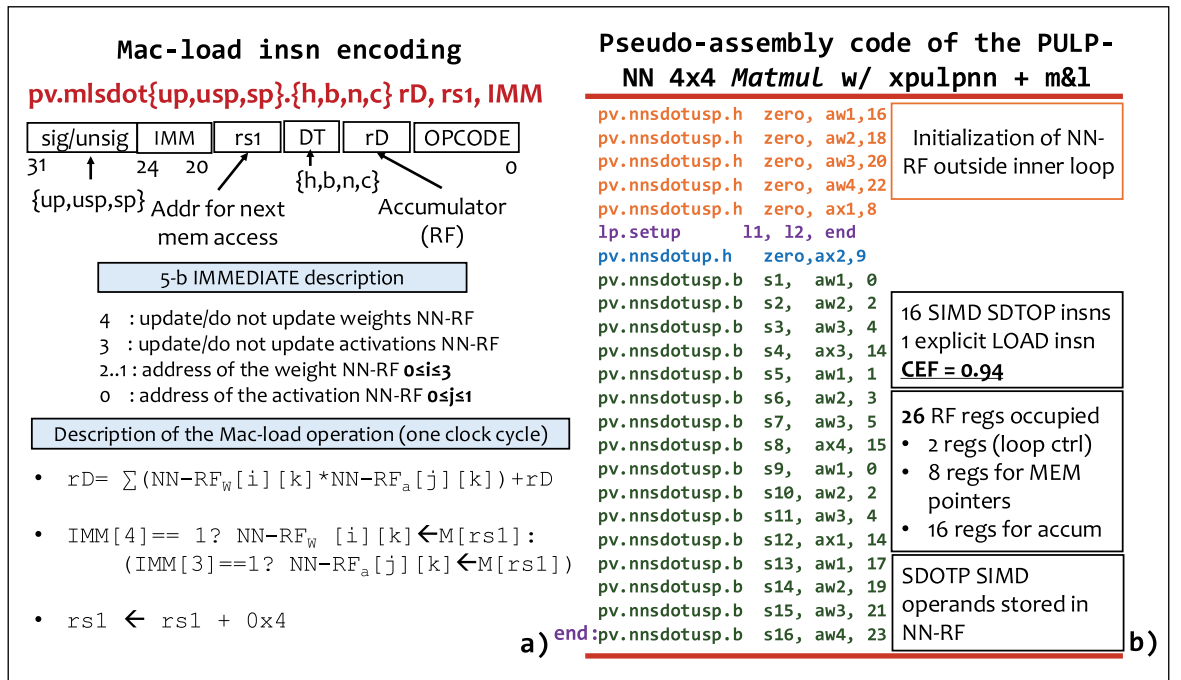


Figure 2. (a) Encoding of the *m&l* instruction, the description of how the immediate field allows to control the NN-RF, and the description of the operations performed by the *m&l*. (b) MatMul innermost loop of the PULP-NN library implemented by exploiting the *m&l*, highlighting the utilization of the GP-RF.

Xpulpnn 4-b and 2-b SIMD instructions	Description of the instruction i=0..8 (4-b); i=0..15 (2-b)	Pseudo-assembly code of the PULP-NN 4x2 Matmul w/ xpulpnn
ALU SIMD INSTRUCTIONS		lp.setup11, 12, end# HW loop setup p.lww1, 4(aw1!)# Load word insn p.lww2, 4(aw2!)# with p.lww3, 4(aw3!)# post-increment p.lww4, 4(aw4!)# of the pointer p.lwx1, 4(ax1!) p.lwx2, 4(ax2!) pv.sdotusp.b/n/cs1, x1, w1# 8/4/2-b SIMD pv.sdotusp.b/n/cs2, x1, w2# sdotp pv.sdotusp.b/n/cs3, x1, w3# instruction pv.sdotusp.b/n/cs4, x1, w4# one clk cycle pv.sdotusp.b/n/cs5, x2, w1# latency pv.sdotusp.b/n/cs6, x2, w2 pv.sdotusp.b/n/cs7, x2, w3 end:pv.sdotusp.b/n/cs8, x2, w4
pv.add[.sc].{n, c}	rD[i] = rs1[i] + rs2[i]	
pv.sub[.sc].{n, c}	rD[i] = rs1[i] - rs2[i]	
pv.avg(u)[.sc].{n, c}	rD[i] = (rs1[i] + rs2[i])>>1	
SIMD VECTOR COMPARISON		
pv.max(u)[.sc].{n, c}	rD[i] = rs1[i] > rs2[i] ? rs1[i] : rs2[i]	
pv.min(u)[.sc].{n, c}	rD[i] = rs1[i] < rs2[i] ? rs1[i] : rs2[i]	
SIMD VECTOR SHIFT OPERATIONS		
pv.srl[.sc].{n, c}	rD[i] = rs1[i] >> rs2[i] (logic shift)	
pv.sra[.sc].{n, c}	rD[i] = rs1[i] >> rs2[i] (arith shift)	
pv.sll[.sc].{n, c}	rD[i] = rs1[i] << rs2[i]	
SIMD VECTOR ABSOLUTE VALUE OPERATION		
pv.abs.{n, c}	rD[i] = rs1[i] < 0 ? -rs1[i] : rs1[i]	
DOT-PRODUCT AND SUM-OF-DOT-PRODUCT OPS		
pv.dot{u,s,us}.{n, c}	rD = +rs1[i]*rs2[i]	
pv.sdot{u,s,us}p.{n, c}	rD = +rs1[i]*rs2[i] + rD	
{n,c}: nibble → 4-b SIMD, crumb → 2-b SIMD		
{u, s, us}: uint, int, 1st operand uint – 2nd op int		
a)		Wi Xi : weights/act.22 GP-RF regs occupied: elements- 6 regs for pointers AWi AXi : MEM pointers- 8 regs for accum. Si : accumulators- 6 regs for data Li : loop setup- 2 regs for loop ctrl b)

Figure 3. (a) Overview of the xpulppnn instructions for nibble (4-b) and crumb (2-b) SIMD vectors. (b) Pseudoassembly code of the 4x2 MatMul implementation of the PULP-NN library, exploiting the xpulppnn instructions, including the GP-RF resources needed to execute the kernel.

microarchitecture. To support this, the RF should handle three simultaneous writebacks: the *dotp* result, loaded memory data, and updated memory address. However, a three-write-port RF is costly in terms of area and power for low-power devices.

To minimize costs, *m&l* specifies the operations on a separate neural network register file (NN-RF) instead of relying solely on the GP-RF. The instruction encodes NN-RF operations using a dedicated 5-bit immediate field, which specifies two NN-RF partitions: a two-register slot for QNN activations and a four-register slot for QNN weights. Three bits define the partitioning, while 2-bit control updates to these partitions. If the two bits are not set, the instruction only performs *dotp/sdotp* on NN-RF operands, with no NN-RF updates and with the third scalar operand sourced from the GP-RF.

This design offers two key advantages: 1) reduced implementation cost, as reducing the GP-RF write ports reduces area and power overhead, while the additional NN-RF is small and 2) enhanced data reuse since NN-RF operands can persist across iterations, lowering memory traffic and providing flexibility in operand management, independently of

compiler scheduling. The programmer can explicitly control data reuse in C code.

Figure 2b illustrates the *MatMul* implementation using *m&l*. Initialization of the NN-RF, performed outside the innermost loop, minimally impacts performance. With operands stored in the NN-RF, the GP-RF handles only accumulators, loop control variables, and memory addresses. This enables a higher data reuse factor (“4 × 4” structure), achieving CEF up to 0.94.

Mixed-precision instructions

Many studies, including [4], highlight the effectiveness of mixed-precision integer quantization in compressing CNN models with minimal accuracy loss, often outperforming uniform schemes. However, if not natively supported by the ISA, mixed-precision operations introduce overhead. For instance, in an 8 × 4 b operation on an ISA limited to uniform-precision SIMD support, software must extract and cast 4-b elements (via bit-masking and extension) to match 8-b precision before performing a *dotp* operation, as illustrated in Figure 4b-2. This process repeats for the upper four elements in subsequent iterations, causing additional inefficiency.

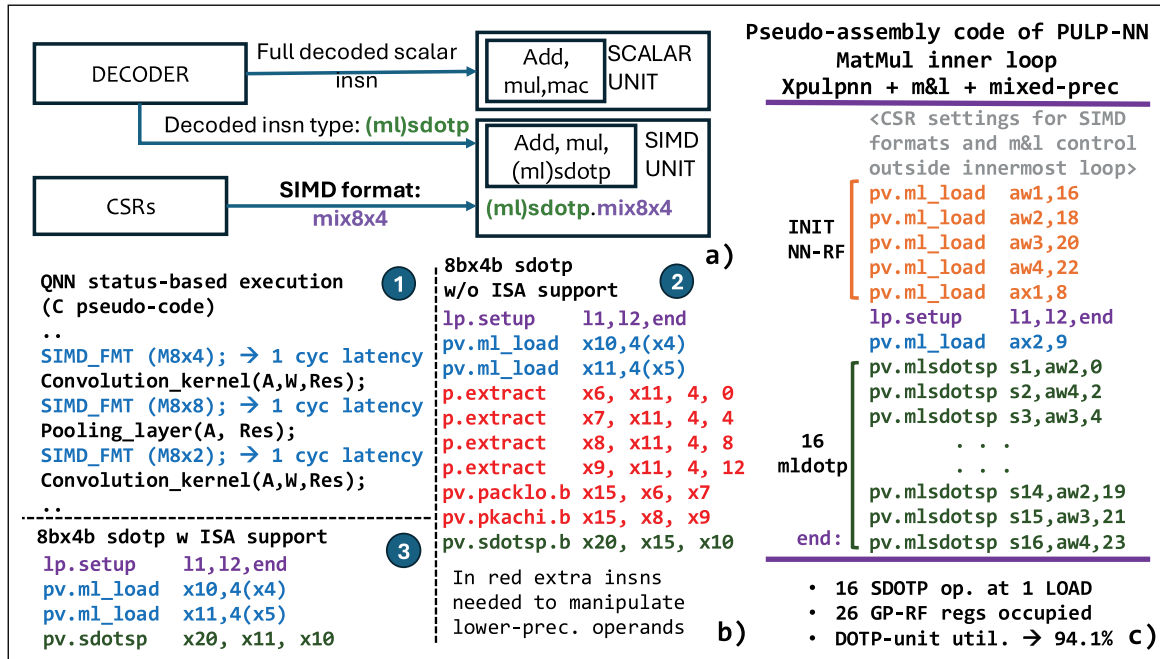


Figure 4. (a) Working principle of the status-based execution of SIMD instructions. (b) Usage in C code of status-based features and the advantage of the mixed-precision support in the ISA comparing two kernels with and without mixed-precision instructions enabled. (c) Pseudocode of the 4 × 4 MatMul implementation of the PULP-NN library, exploiting mixed-precision status-based operations.

To address this issue, we extended *xpulpnn* to support mixed-precision SIMD operations in hardware. Encoding all potential mixed-precision formats in the *xpulpnn* instruction fields, however, would require over 100 additional instructions for *dotp* operations alone, consuming significant RISC-V encoding space and complicating microarchitectural decoding. Instead, we adopted a status-based execution flow, similar to vector architectures, as a lightweight solution.

Figure 4a illustrates the approach. A control and status register (CSR) enables/disables the status-based mode. When disabled, instructions follow the standard *xpulpnn* specification. When enabled, the instructions define operations and operands, while their precision formats are specified via another CSR. This register, programmable in one clock cycle, allows the programmer to set the desired precision configuration before starting the SIMD computation, ideally outside the kernels' innermost loop. During execution, handling vector slicing is delegated to hardware, reusing different chunks of the lower p_1 -precision vector over D higher p_2 -precision vectors, where $D = p_2/p_1$.

This method enables hardware-accelerated mixed-precision TinyML kernels without adding

new instructions to the *xpulpnn* ISA or incurring performance overhead. Since *dotp* formats change layer-by-layer, the single-cycle CSR write latency is negligible compared to the thousands of cycles required for a *MatMul* or convolution layer. Figure 4b and c demonstrates how programmers set SIMD precision in C code and highlights the benefits of mixed-precision SIMD operations. Without this support, casting lower-precision operands to match high-precision operands within kernel loops adds significant overhead, degrading performance.

Microarchitecture

We implemented the proposed ISA extensions in a four-stage, in-order, single-issue RISC-V pipeline and analyzed the performance and costs. The baseline core, RI5CY [12], featuring the RV32IMCP ISA, supports uniform SIMD operations on 16- and 8-bit vectors for integer DSP. Figure 5 outlines the necessary modifications to support the ISA extensions for TinyML.

To enable uniform *xpulpnn* ISA and subbyte SIMD operations, we extended the decoder, ALU, and *dotp*-unit, resulting in a 5-kGE (8.33%) area increase over RI5CY. The updated core, *MicroSIMD*,

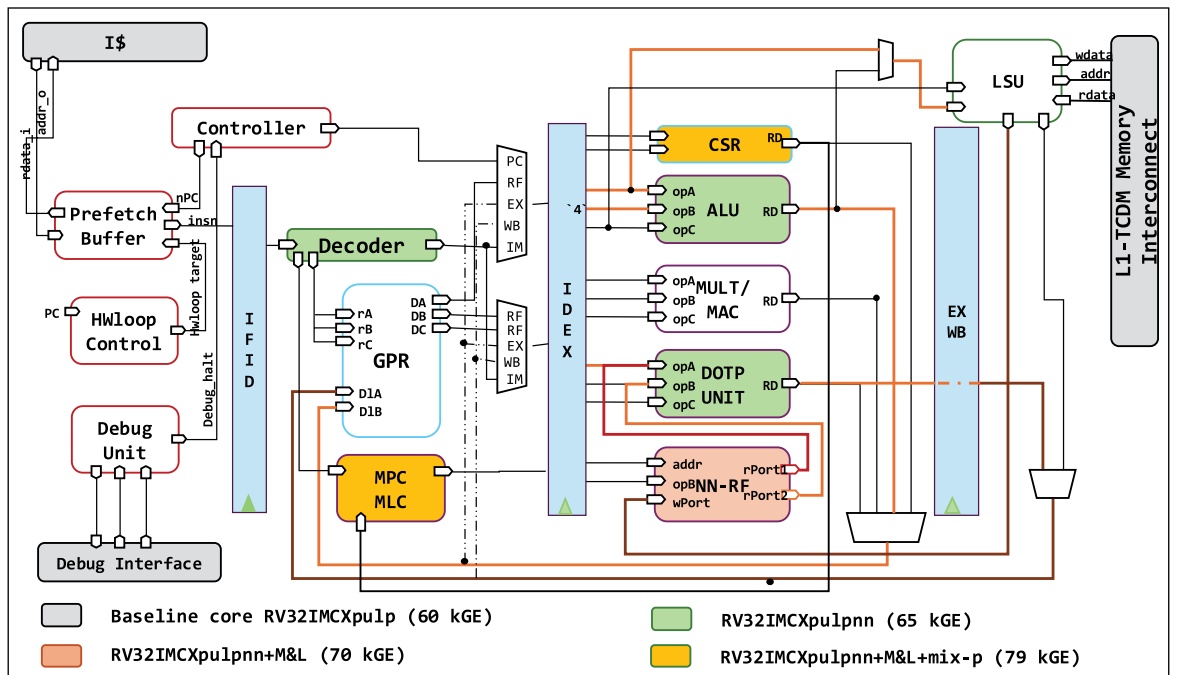


Figure 5. Architectures of the four-stage single-issue in-order RISC-V pipeline, implementing the RV32IMCXpulp ISA and, incrementally, all the proposed additional ISA extensions. Area in kilogate equivalent (kGE) extracted from physical implementation.

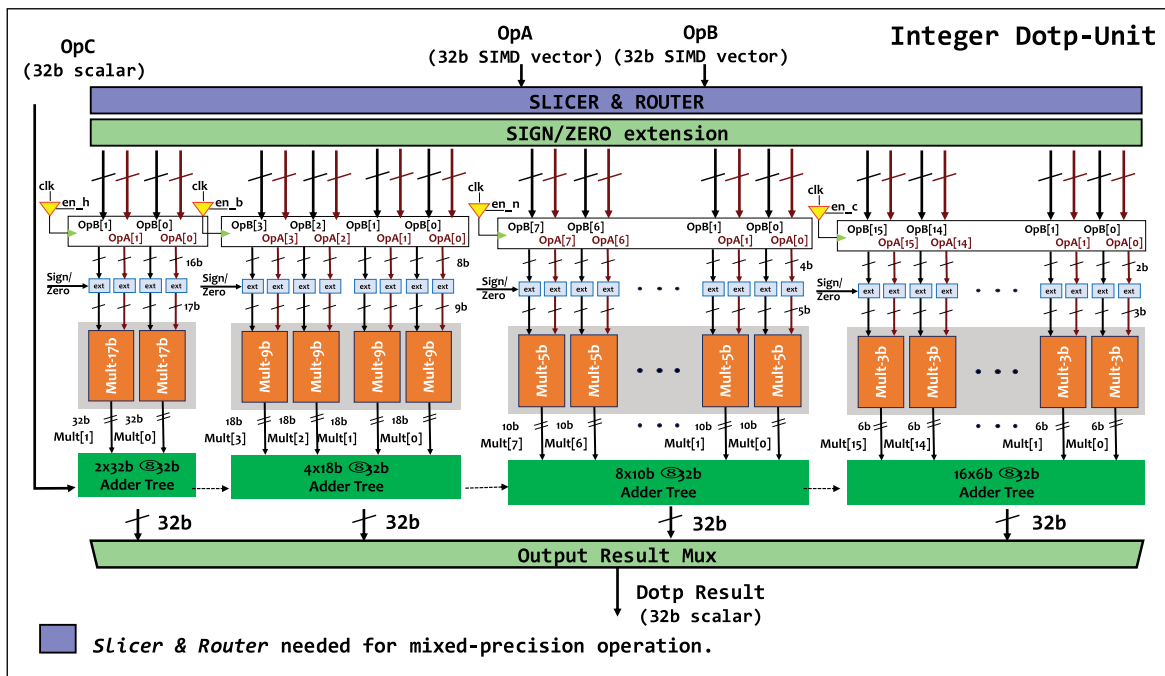


Figure 6. Block diagram of the proposed dotp unit. To support xplupnn SIMD operations, we add extra 8×4 and 16×2 bit multipliers in corresponding bitwidth regions. To support mixed-precision operations in hardware, we add extra logic at the input of the multipliers to slice, route, and sign-zero-extend the operands.

incorporates an extended dotp-unit, as detailed in Figure 6. This unit performs single-clock-cycle Dotp operations between two SIMD registers, accumulating partial results into a 32-b scalar register. The SIMD input vectors are symmetric in precision and can contain two 16-bit, four 8-bit, eight 4-bit, and 16 2-bit elements. To support both signed and unsigned operations, the inputs to the multipliers are 1-bit sign/zero-extended. The dotp of two (N)-bit SIMD vectors is computed within a dedicated (N)-bit region. Each region includes a number of multipliers equal to the vector elements, followed by an adder tree that sums the partial products. This design avoids intermediate registers, ensuring that results are directly accumulated into the scalar register within a single clock cycle. The SIMD MAC operation, *sdotp*, is executed by adding another 32-b scalar operand to the input of the adder tree.

We opted for distinct hardware regions for each bitwidth, rather than sharing or reconfiguring a single SIMD multiplier, to minimize latency impacts on the processor's critical path, which involves the path from the processor to the data memory and vice versa.

The *dotp*-unit is close to being time-critical due to the partial multiplications and the adder trees executed combinatorially in the same clock cycle. Therefore, adding additional logic to select, split, and manipulate inputs before feeding the single SIMD unit, as well as additional logic to reconfigure the SIMD unit to operate at the desired precision, would be detrimental from the timing viewpoint. In contrast, adding pipeline registers to a single reconfigurable SIMD unit would cause execution stalls when computing data-dependent back-to-back operations, resulting in a significant performance drop in QNN convolution kernels, which are mainly characterized by sequences of back-to-back *sdotp* operations, as reported in “QNNs execution model.”

As a consequence of our design choice, the area overhead accounts for 18.4% of the baseline EX-stage of the processor. However, this is not reflected by a similar power overhead at the core level, which is negligible as shown in “Physical implementation.” This is the result of clock gating and operand isolation, which prevent switching of unused datapath sections. A similar design rationale holds for the

extension of the ALU. They are omitted for the sake of conciseness.

To support also the *m&l sdotp* operation, the hardware costs consist of the additional NN-RF with 6 32-bit registers, one write and two read ports, integrated into the EX-stage of the processor's pipeline, and additional logic to distribute the operands from the NN-RF to the dotp unit and the LSU to the NN-RF, while no additional arithmetic block is required other than the ones already present in the *MicroSIMD* core. The extended core, namely, *NeuroSIMD*, brings another 5-kGE cost (20% extra area) over the *MicroSIMD* core, which means an overall 16.7% overhead compared to the baseline.

From a power consumption viewpoint, the m&l has a limited, but nonnegligible impact due to the additional NN-RF. To avoid unnecessary switching activity, we applied clock gating on the NN-RF. With such techniques, the power overhead of *NeuroSIMD* compared to the baseline core remains below 10%, as discussed in “Physical implementation.”

To support mixed-precision operation execution, we extended the *NeuroSIMD* pipeline with extra CSR registers for storing the operations' formats, we added a mixed-precision controller (MPC) in the ID stage to handle the selection, slicing, and routing of SIMD vector elements, and we extended the dotp unit, which is preceded by a slicer&router (S&R) network, controlled by the MPC, which slices the registers according to the format (FTM) specified by the MPC.

In particular, in mixed-precision operations, we have operands of different-sized elements, with the lower-precision SIMD vector (that we call *RS2*) containing more vector elements, as a consequence. During the operation, one register is fully utilized (the one with high-precision elements that we call *RS1*), while only a part of *RS2* is needed.

In the proposed implementation, the S&R selects the subportion of the second input register (*RS2*) to be used in the current operation and sign-/zero-extends it to match the size of vector elements in the input register *RS1*. Afterward, the network routes the two manipulated vectors (now featuring the same elements' precisions) to the appropriate set of multipliers (e.g., an 8×4 b *dotp* requires 8-bit SIMD multipliers).

To minimize the logic necessary to implement the mixed-precision operations, we constrained the first operand *RS1* to always feature high-precision

elements without loss of generality given the commutative property of integer add-and-multiply operators.

The resulting pipeline, namely, *NeuroMixed* core, shows extra 9-kGE costs over *NeuroSIMD*, with 32% of area overhead compared to the baseline. A full analysis of the costs is reported in “Physical implementation.”

Vector lockstep parallel execution

In this section, we describe how we integrated the cores extended with the new instructions into a parallel compute cluster, and we discuss a novel reconfigurable design at the cluster level that allows the cluster to operate either in classic MIMD mode on control tasks or in the energy-efficient VLEM when executing instructions-regular kernels such as *MatMuls*.

To exploit the parallelism of QNN workloads (convolutions and *MatMuls*), the RISC-V cores discussed in “RISC-V ISA extensions for TinyML” are integrated into an *N*-core PULP cluster for parallel execution, expecting a near-linear scaling of the performance when moving from single-core to multicore contexts [10]. The PULP cluster normally operates in the MIMD mode, i.e., each core fetches, decodes, and executes instructions independently.

However, this execution mode is suboptimal for the execution of the innermost loop of QNN convolutions. Thanks to the *im2col*-based computing paradigm, which, combined with SIMD loads and HWC data layout, is the state-of-the-art approach on TinyML devices for efficient data movements, the innermost loop of QNN convolutions becomes a *MatMul*. On a parallel cluster, executing the *MatMul* implies that each core performs the same operation (*sdotp*) over different data [10]. In an MIMD execution model, this would require extra unnecessary energy to fetch the same instruction for all the cores at the same clock cycle.

To reduce the energy consumption not spent in computation, we introduced a software-configurable hardware mechanism at cluster level, as depicted in Figure 7, which allows the cluster to operate either in MIMD or VLEM: in the latter case, one of the cores acts as *leader* (core 0) and is the only one to fetch instructions; the other cores, *followers*, decode and execute instruction bits received from the *leader* core, reducing the number of equal instructions to be fetched from instruction caches (*I\$*). Due to the

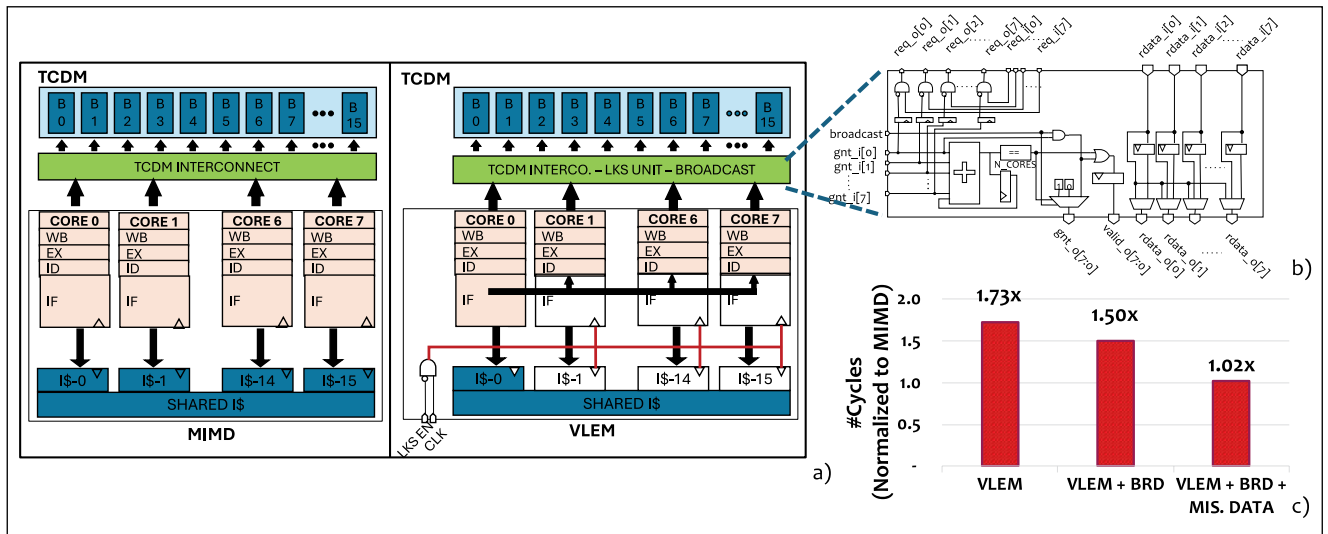


Figure 7. (a) Architecture of the re-configurable parallel ultralow-power (PULP) cluster: on the left, executing in MIMD mode; on the right, executing in VLEM, with IF stages and private instruction caches of *follower* cores clock-gated to save power. (b) Details of the lockstep interconnect and the broadcast unit. (c) Performance of the VLEM with and without broadcasting and data misalignment enabled.

inactivity of fetch stages and private *I\$* of *follower* cores, these are clock-gated when in VLEM to further save dynamic power.

To enter the VLEM, the cores must: 1) synchronize on a barrier and 2) write to a memory-mapped register. This procedure has an impact on the instruction cache subsystem: the *leader* core's private cache is untouched (being the core that will continue fetching instructions), but the caches of the *follower* cores terminate any in-flight cache refill and are then clock-gated. At the end of the VLEM section in the program code, a store operation to a specific register unlocks the cores, starts a hardware procedure to restore the cache state, and resumes the execution in MIMD. In particular, the program counter of the *follower* cores will be set equal to the one of the *leader* core. As a consequence, depending on the number of instructions executed in VLEM, resuming in MIMD can cause the *follower* cores to stall immediately due to cache misses.

During the VLEM mode, all cores must be kept synchronized. Bank conflicts in the *L1* memory are sources of desynchronization. To solve the conflicts when in MIMD mode, the logarithmic interconnect of the cluster serializes concurrent load/store operations from different cores to the same memory bank. This is done by exploiting the request/

grant handshake of the interconnect: if the grant for a given core is not asserted, the core stalls until it arrives. We extend this approach to the VLEM with additional minimal logic: upon concurrent conflicting requests, all the cores remain stalled until all the memory accesses are completed, after which the execution can safely restart in the lockstep mode at the system level. While this approach does not result in a performance drop when in MIMD since the cores execute independent instruction flows, it introduces significant overhead when in VLEM, as shown in Figure 7c.

A common case is when cores access the same word in the same memory bank, e.g., a pointer to the base of a shared array, such as weight filters in a DNN. To solve conflicts from cores accessing the same *L1* address in memory when in VLEM, we designed a broadcasting unit (BU) and integrated it into the logarithmic interconnect: it snoops and compares request addresses from the cores, and in the case of an address match, it forwards only one memory request to the memory system; upon response from the memory, the BU sends the received data to all the cores, in one clock cycle.

Concurrent accesses to the same memory banks, but different addresses, are not solved by the BU. These typically are systematic and occur when the

size of memory structures is multiple of the number of banks, common in arithmetic-intensive kernels such as QNN convolutions. A simple yet effective countermeasure is to allocate data so that the base addresses seen by each core are never aligned in the same bank. A combination of these hardware/software techniques allows eliminating conflicts overhead, as shown in Figure 7c.

Programming model

To control in software the hardware extensions proposed, we extend the PULP GCC compiler to allow the usage of *xpulpnn* RISC-V instructions in the C code through the explicit invocation of *intrinsics* and a set of a hardware abstraction layer (HAL) to facilitate VLEM programming, including a set of primitives for core identification, synchronization, and memory allocation.

Since the SIMD vectors do not comply with the GNU vector extensions and since GCC does not handle data formats of less than eight bits, we represent SIMD vectors as 32-bit opaque data types (i.e., `int32_t`) and define a set of intrinsics in the GCC back end for the mixed-precision operations supported. This method, unlike assembly inlining, generates high-level intermediate representations (IR) that the compiler uses for target-specific optimization passes to maximize the reuse of operands and efficient scheduling of instruction flow. Although programmers cannot directly manage NN-RF registers since the updates are hidden in C, the back-end IR tracks these relations, aiding optimization.

These low-level descriptions are integrated into optimized software primitives, namely, *PULP-NN*, which are exposed to the embedded programmer as APIs and invoked in C as library functions or integrated as a back-end library into QNN end-to-end deployment tools [27].

Experimental setup and results

In this section, we evaluate the proposed enhancements at the core and cluster levels from a physical viewpoint and from a performance and energy efficiency perspective. First, we examine the microarchitectural costs of implementing the RISC-V ISA instructions, focusing on area, power, and timing overheads compared to the RI5CY baseline. To provide incremental insights on the costs of implementing extensions, we consider

four different versions of the core: the baseline RI5CY, *MicroSIMD* implementing the RV32IMCpxpulpnn, *NeuroCore* implementing RV32IMCpxpulpnn+m&l, and *NeuroMixed* implementing RV32IMCpxpulpnn+m&l+mixed-precision.

To this purpose, we implement all the versions in Global Foundries 22-nm FDX technology, down to a silicon-ready layout through logic synthesis with Synopsys Design Compiler and full place-&route via Cadence Innovus. We target the worst-case corner (SS, 0.59 V, $-40^{\circ}/125^{\circ}$). To extract power numbers, we analyze VCD traces from timing-annotated postlayout simulations using Synopsys Prime Time. Power analysis is performed in low-voltage typical corners (TT, 0.65 V, 25°), simulating normal operating conditions at room temperature.

To evaluate our solution at the cluster level, we use the same methodology. We integrate the *NeuroMixed* core into an octacore parallel cluster in two different versions. The first, our baseline for the cluster, corresponds to the system-level architecture introduced in “[Parallel Ultra-Low-Power Cluster](#),” without any additional features. The second version implements the proposed vector lockstep feature that allows the cluster to operate in MIMD mode or VLEM.

Second, we assess the performance and energy efficiency gains from the proposed microarchitectural solutions for the innermost loop of QNN convolutions. We benchmark the cores and the cluster using the PULP-NN library [10], which offers optimized QNN kernels for both single-core and parallel execution over the eight cores of the cluster. The C kernels of PULP-NN are extended to address sub-byte and mixed-precision convolutions, inferring new instructions in the program code, as outlined in “[Programming model](#).”

To demonstrate efficiency improvement in key kernels for QNN inference, we choose layers that fit within L1 memory, thus avoiding transfer overhead. These layers represent common tiles employed in QNN deployment frameworks [27] utilizing PULP-NN as the compute back-end library. The evaluated layers operate on a $32 \times 32 \times 32$ input tensor with a $64 \times 3 \times 3 \times 32$ filter.

To contextualize our results, in “[Comparison with state-of-the-art](#),” we compare our optimized solution with leading software and hardware for microcontroller devices in the same power class, including ARM-based solutions available commercially.

Physical implementation

Table 1 compares the cores in terms of timing, area, and power consumption. Power measurements are performed on an 8-bit matrix multiplication workload and a GP application consisting of a mix of RV32IMC arithmetic and control instructions.

The *MicroSIMD* core shows a 10% area overhead over the baseline core, primarily due to the extension of the *dotp* unit and the ALU to support subbyte SIMD operations. The *m&l* operation in *NeuroCore* adds a 7% of area overhead over *MicroSIMD*, stemming from the NN-RF and its control logic, and slightly reduces operating frequency due to extra gates on the critical *m&l* datapath. The *NeuroMixed* core supports mixed-precision operations and *m&l* in hardware, occupying an area of 0.03 mm², with a 32% area overhead over the baseline, with a 2% operating frequency reduction, due to added logic for slicing and routing the operands at the input of the *dotp* unit. Notably, the area overhead of *NeuroMixed* is less impactful at the cluster level, showing only a 2% overhead compared to a cluster with baseline RI5CY cores.

In terms of power consumption, due to the power-aware design discussed in “RISC-V ISA extensions for TinyML,” the overhead over the baseline remains below 10% at the core level, and it reduces to under 5% when we consider the overall cluster system. This holds also on GP applications that do not leverage the new instructions, where we observe a power overhead of 5.8%. These results demonstrate the effectiveness of ISA

extensions combined with power-aware microarchitectural design to specialize instructions processors without compromising energy efficiency on GP workloads.

Supporting dynamic execution modes such as MIMD and VLEM increases the power overhead by only 3% on GP applications. At the same time, it substantially reduces the power consumption by 35% when running MatMuls in VLEM mode since VLEM lowers switching activity in private caches and fetch stages using clock gating and reduces memory traffic through the logarithmic interconnect, thanks to the *broadcasting* feature.

Benchmarking

To evaluate the performance and energy efficiency gain achieved with the proposed ISA extensions, as well as the VLEM feature at the cluster level, we benchmark the innermost loop of convolution kernels of PULP-NN, *MatMuls*, in different bitwidth configurations, including mixed-precision formats from 8 down to 2 bits.

The kernels are implemented using the PULP-NN library in single-core and multicore configurations. On the baseline core, subbyte or mixed-precision kernels manipulate operands in software (through casting and bit-manipulation functions) to match the compute precision supported by the hardware. On extended cores, the kernels are extended to exploit ISA extensions and optimize the *MatMul* layout (as discussed in “RISC-V ISA extensions for TinyML”), wherever feasible, to maximize data reuse at the

Table 1. Area, power, and timing evaluation of the cores and clusters.

	Area [um ²] Ovh. vs Baseline [%]	Power @ 400 MHz [mW] Ovh. vs. Baseline [%] GP workload	Power @ 400 MHz [mW] Ovh. vs. Baseline [%] 8-bit MatMul	Max op. fre- quency [MHz]
Baseline (RI5CY)	24300	0.71	1.18	420
MicroSIMD	26008 (10%)	0.70 (-1.7%)	1.10 (-1.7%)	420
NeuroCore	27821 (17.9%)	0.71 (0.4%)	1.21 (7.7%)	415 (-1.1%)
NeuroMixed	31300 (32.7%)	0.72 (1.2%)	1.27 (8.4%)	412 (-1.9%)
Parallel Cluster of 8 cores				
Baseline w/ RI5CY	723252	12.9	19.3	400
Baseline w/ Neuro- Mixed (MIMD)	763604	13.4	21.5	400
Reconfigurable MIMD/VLEM	806791 (5.7%)	13.8 (3%)	14.1 (-34.5%)	400

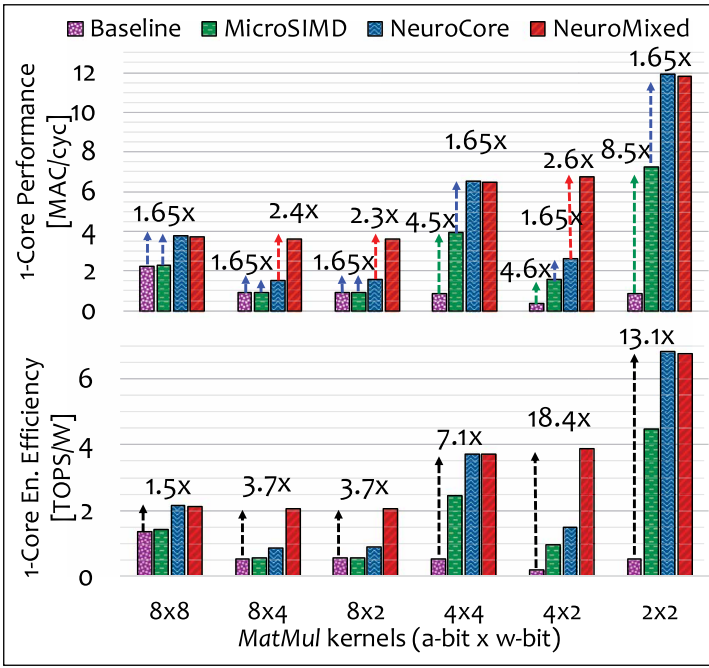


Figure 8. Single-core execution performance (in MAC/cycle) and energy efficiency (at 0.65 V, 400 MHz, 25° C) of PULP-NN convolution's innermost loop (*MatMul*). The kernels, at various operand precisions, exploit the ISA extensions available on the target cores. The *Baseline* is the *RI5CY* core.

RF level and increase the computation efficiency. The measurements are conducted at the maximum achieved core operating frequency, in the TT corner, at 0.65 V and 25 °C room temperature. Results for a single-core execution are shown in Figure 8.

First, we observe that while the *xpulpnn* ISA implemented by the *MicroSIMD* core significantly enhances performance and energy efficiency by 8.5× on symmetric subbyte kernels compared to the baseline core, these extensions do not address the overheads associated with mixed-precision kernels. Second, we observe that the *m&l* instruction implemented by *NeuroCore* leads to significantly improved performance and energy efficiency across all kernel types considered compared to the baseline core. Thanks to *m&l*, it is feasible to perform matrix–matrix multiplications with a higher data reuse factor (as detailed in “RISC-V ISA extensions for TinyML”), which enhances *dotp* unit utilization to as high as 0.94. However, this improvement is limited in mixed-precision scenarios, where the benefits of *m&l* are offset by the additional instructions needed to cast the data prior to executing the *dotp*. Ultimately,

the fully featured *NeuroMixed* core, although it incurs nonnegligible area and power costs, provides the highest peak performance and energy efficiency, achieving improvements of 14.2× and 12.5×, respectively, compared to the baseline core.

These results are reflected at the cluster level, where the optimized software routines of PULP-NN can achieve a near-to-ideal speed-up when we move from single-core to multicore execution. Results are reported in Figure 9. To evaluate the advantages of the proposed VLEM on regular computing kernels such as *MatMuls*, we measure the performance and energy consumption with the cluster running the *MatMuls* in two modes: the classic MIMD mode and VLEM. On the one hand, we notice that the performance loss introduced by the VLEM is negligible. On the other hand, the energy efficiency boost is significant, up to 1.6× compared to the cluster in MIMD mode. This points out the MIMD/VLEM reconfigurable cluster architecture that we propose as a low-cost but very effective solution to boost the energy efficiency of regular kernels such as matrix–matrix multiplication, without incurring the overhead to handle divergent branches or control applications, which can still be performed in MIMD mode, contrarily to GPUs, which can only execute in SIMD mode.

Comparison with state-of-the-art

To put our solutions in perspective, we compare our results with state-of-the-art existing programmable hardware and software solutions for TinyML in terms of performance and energy efficiency. To carry out the comparison, we run QNN convolution layers also on two off-the-shelf commercial microcontrollers, STM32H7⁵ and STM32L4,⁶ integrating a single ARM Cortex-M7 and a single ARM Cortex-M4 core, respectively. To execute subbyte and mixed-precision kernels and extract performance numbers in terms of MAC per cycle, we use the extended CMSIS-NN [8]. In addition, we compare our results with the newest ARM Cortex-M55 core, which implements the Helium ISA [9]. We rely on performance and energy efficiency numbers reported in the whitepaper [29], which claims a 5× speed-up in AI CMSIS-NN kernels compared to the Cortex-M7, with a 7× improvement in energy efficiency.

⁵<https://www.st.com/resource/en/datasheet/stm32h743bi.pdf>

⁶<https://www.st.com/resource/en/datasheet/stm32l476je.pdf>

Figure 10 summarizes the results. From an architectural comparison at the core level, the *NeuroMixed* core outperforms the performance of the Cortex-M4 by 8× to 33× across various precision kernels, primarily because of the limited SIMD MAC support in the Armv7E-M ISA of the Cortex-M4, which only supports 16-bit operations in hardware. Compared to the Cortex-M7, *NeuroMixed* still performs better (5× to 24×), despite that the Cortex-M7 implements the same Armv7E-M ISA but with a dual-issue in-order pipeline capable of executing up to two instructions per cycle. Against the Cortex-M55, which supports 8-bit SIMD MAC operations via Helium ISA, *NeuroMixed* shows comparable performance for symmetric 8-bit kernels, but it performs up to 5× better on mixed-precision subbyte cases. At the cluster level, our eight-core cluster adds another ~8× improvement factor, leading to a peak of 29 and 91 MAC/cycle on 8- and 2-b *MatMuls*, respectively, outperforming the Cortex-M55 by 7.2× and 34.3×.

In terms of energy efficiency, we compare against a high energy-efficient implementation of the Cortex-M4 core [28], in the same GF 22-nm technology node that we used. To this extent, we project the performance (in MAC/cycle) obtained by benchmarking the STM32L4 in the best energy efficiency point of [28]: this is achieved through adaptive body biasing at 0.46 V and 190 MHz, with a power density of 6.48 $\mu\text{W}/\text{MHz}$. Despite that our system does not integrate any body biasing, the eight-*NeuroMixed*-core VLEM cluster still shows 12× better energy efficiency than the Cortex-M4 core on 8-b *MatMul* kernels and up to 46× on 2-b ones.

The high performance and power proportionality of our VLEM cluster stem from architectural and technological factors. These include hardware support for SIMD dotp operations with 8-, 4-, and 2-bit precision, including all mixed-precision variants, combined with the m&l instruction, as well as the use of a computing cluster with eight processors that can operate in VLEM and its implementation in an advanced technology node for TinyML systems, significantly enhancing computational throughput and energy efficiency compared to the single-core design of Cortex-M based systems.

Overall, this analysis demonstrates that ASIC-like energy efficiency for QNN workloads is achievable on fully programmable, tiny MCU systems at the extreme edge of IoT. This outcome is made possible by combining domain-specific RISC-V ISA

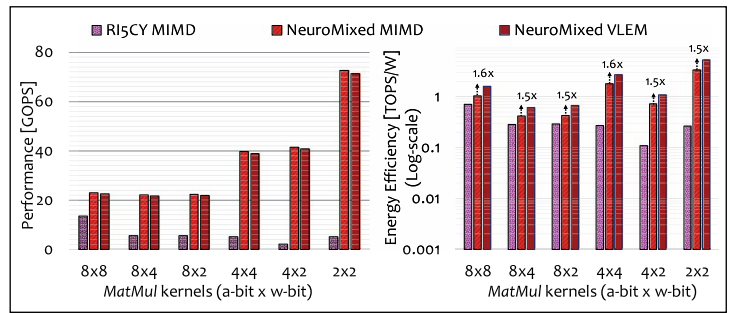


Figure 9. Eight-core execution performance and energy efficiency (at 0.65 V, 400 MHz, room temperature) of PULP-NN convolution’s innermost loop (*MatMul*). The kernels run on an eight-core parallel cluster using the extended PULP-NN library. The cluster integrates the RI5CY core in *RI5CY MIMD* configuration, while it integrates *NeuroMixed* processors and runs in *MIMD* mode or *VLEM*, *NeuroMixed MIMD*, and *NeuroMixed VLEM*, respectively.

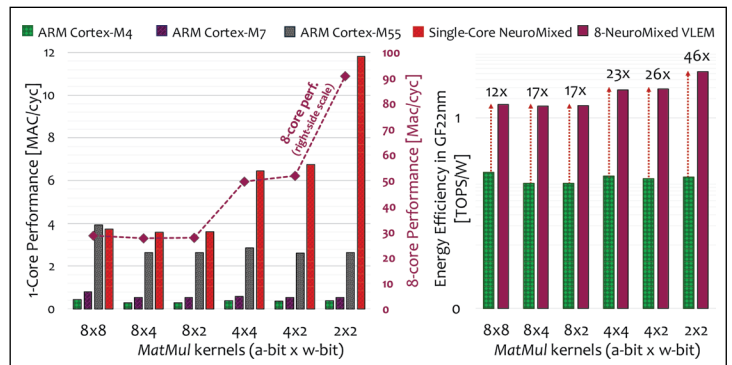


Figure 10. Comparison with state-of-the-art ARM Cortex-M processors for TinyML. The left graph shows a single-core execution performance comparison, and reports peak performance achieved on our eight-core cluster. The right graph compares energy efficiency between our eight-core VLEM cluster (0.65V, 25 °C, max freq. 400 MHz) and an energy-efficient Cortex-M4 implementation in 22-nm technology [28] (0.46V, 25 °C, max freq. 190 MHz).

extensions, power-aware microarchitecture design, and multicore cluster integration, leveraging cutting-edge near-threshold FDX technology.

Discussion

The design presented in the previous sections is highly efficient for small TinyML systems; however,

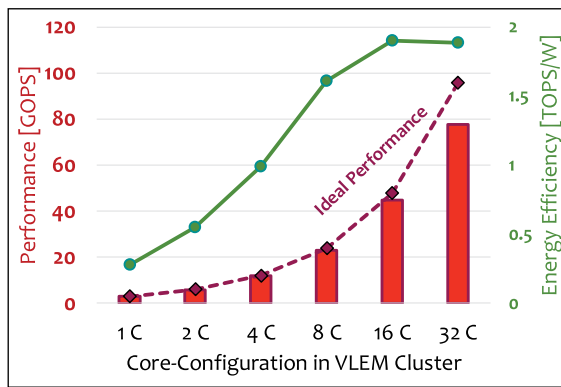


Figure 11. Scalability study on core configuration for the VLEM cluster in terms of performance and energy efficiency on an 8-bit MatMul kernel.

it does not scale well to larger edge architectures. Our experiments, as summarized in Figure 11, show that power-performance proportionality decreases when the number of cores in the clusters goes beyond 16. The reasons are twofold: the performance bottleneck moves at the core's boundaries since the load/store and fetch-decode paradigm of processors, as well as the limited 32-bit data port toward the scratchpad memory, make it challenging to keep the cores' datapath busy with data; moreover, as we scale up with the number of cores, the single-cycle latency interconnect significantly grows in size and increases the power consumption of the overall compute cluster, with visible diminishing energy efficiency.

To scale up performance at limited area costs, a possible solution is to tightly integrate specialized accelerators in the clusters as RISC-V coprocessors. To minimize area and power impact, these accelerators can be designed with standardized hardware processing engine (HWPE) interfaces [11] with data ports $\gg 32$ -b wide. This interface features minimum-sized internal buffers to guarantee close to 100% datapath utilization, while they rely on the shared L1 memory as the primary buffer. The configuration plane of the accelerators can be controlled by the RISC-V core through ISA extensions or memory-mapped registers. Finally, a two-level hierarchical heterogeneous interconnect (HCI) [11] guarantees low-latency high-bandwidth access toward the memory while minimizing memory conflicts between cores and accelerators, ensuring higher scalability before hitting inefficiency points.

An alternative emerging approach to enhance processor efficiency is adopting a vector ISA, which promises substantial benefits in terms of performance and energy efficiency. First, vector-based ISA extensions provide improved scalability by enabling parallelism across wide vector lanes. This reduces the pressure on interconnects, which often limits the performance of scalar multicore systems. Second, vector processors can be latency-tolerant architectures through the use of hardware scoreboards and nonblocking load/store interfaces. These mechanisms allow processors to execute instructions independently of memory latency, keeping the datapath active even when memory access stalls are unavoidable. Finally, vector processors achieve significantly higher performance per compute unit compared to scalar 32-bit cores due to the greater data-level parallelism they exploit. This makes them particularly well-suited for AI workloads that involve large datasets and matrix operations.

An early example of vector processor design in the MCU space is Spatz [30], a 32-bit embedded RISC-V vector processing unit (VPU) implementing the Zve32x ISA. Spatz supports 32-, 16-, and 8-bit integer vectors, including mixed-precision formats, and is designed to create compact, efficient, and latency-tolerant processing elements (PEs) in shared L1 memory clusters, tackling the scalability limitations discussed above. A four-lane Spatz cluster achieves 16 MAC/cycle on 8-bit vectors, scaling to 960 MAC/cycle in a 64-cluster system. Similarly, the ARM Cortex-M55 can be configured with an optional vector extension, namely, Helium [9], which processes 64-bit wide vectors with a peak performance of 8 MAC/cycle on 8-bit vector elements, doubling the performance of the scalar core.

While HWPEs and vector engines have a high potential for scaled-up TinyML MCUs, significant challenges still remain in the software ecosystem: compiler support for HWPE offload and autovectorization is not mature for MCUs, and intrinsics are more complex and less widely known than scalar ones, as software optimization for HWPEs and vector processors is not a commonplace skill for embedded programmers.

In this article, we described the RISC-V PULP approach to accelerate TinyML at a low cost. We demonstrated how custom RISC-V ISA extensions enable subbyte and mixed-precision integer arithmetic in hardware, and we discussed a lightweight

m&l instruction to improve the SIMD dotp computing efficiency. Furthermore, we showed how to integrate these extensions in an RISC-V MCU processor core at low extra area costs, and how we exploited power-aware design techniques at the microarchitectural level to reduce power overhead. We demonstrated that this approach can achieve significant performance and energy efficiency gains on TinyML kernels up to 13.9× and 12.5×, respectively, compared to the baseline processor, without reducing its efficiency on GP and control applications.

At the system level, we showed how to integrate ISA-extended processors in an ultralow-power parallel architectural template and detailed how we extended the cluster to operate in VLEM mode. Executing *MatMuls* with VLEM demonstrated a 38% power reduction compared to baseline MIMD execution, leading to up to 1.6× improved energy efficiency on TinyML kernels.

WITH A PEAK performance of 72.6 GOPS and a peak energy efficiency of 5.32TOPS/W in GF 22-nm technology, the cluster significantly outperforms RISC-V and ARM MCUs with only 8-bit SIMD support in terms of performance and energy efficiency. The RISC-V PULP approach establishes that software-programmable TinyML inference on MCUs is possible at efficiency levels close to those achieved by custom accelerators, without compromising flexibility. ■

Acknowledgments

This work was supported in part by the TRISTAN Project through the Key Digital Technologies Joint Undertaking (KDT JU) under Grant 101095947 and in part by the Spoke 1 on Future High-Performance-Computing (HPC) of the Italian Research Center on High-Performance Computing, Big Data and Quantum Computing (ICSC) through the Ministry of University and Research (MUR) for the Mission 4—Next Generation EU Programme.

References

- [1] M. Shafique et al., "TinyML: Current progress, research challenges, and future roadmap," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 1303–1306.
- [2] M. Nagel et al., "Data-free quantization through weight equalization and bias correction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1325–1334.
- [3] A. Zhou et al., "Incremental network quantization: Towards lossless CNNs with low-precision weights," 2017, *arXiv:1702.03044*.
- [4] M. Rusci, A. Capotondi, and L. Benini, "Memory-driven mixed low precision quantization for enabling deep network inference on microcontrollers," 2019, *arXiv:1905.13082*.
- [5] B. Moons et al., "14.5 envision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28nm FDSOI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2017, pp. 246–247.
- [6] J. Lee et al., "UNPU: A 50.6 TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 218–220.
- [7] S. Bavikadi et al., "A survey on machine learning accelerators and evolutionary hardware platforms," *IEEE Des. Test*, vol. 39, no. 3, pp. 91–116, Jun. 2022.
- [8] A. Capotondi et al., "CMix-NN: Mixed low-precision CNN library for memory-constrained edge devices," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 5, pp. 871–875, May 2020.
- [9] ARM. *ARM Helium*. Accessed: Dec. 2024. [Online]. Available: <https://www.arm.com/technologies/helium>
- [10] A. Garofalo et al., "PULP-NN: Accelerating quantized neural networks on parallel ultra-low-power RISC-V processors," *Phil. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 378, no. 2164, Feb. 2020, Art. no. 20190155.
- [11] A. Garofalo et al., "DARKSIDE: A heterogeneous RISC-V compute cluster for extreme-edge on-chip DNN inference and training," *IEEE Open J. Solid-State Circuits Soc.*, vol. 2, pp. 231–243, 2022.
- [12] M. Gautschi et al., "Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 10, pp. 2700–2713, Oct. 2017.
- [13] F. Staudigl, F. Merchant, and R. Leupers, "A survey of neuromorphic computing-in-memory: Architectures, simulators, and security," *IEEE Des. Test*, vol. 39, no. 2, pp. 90–99, Apr. 2022.
- [14] G. Desoli et al., "14.1 a 2.9TOPS/W deep convolutional neural network SoC in FD-SOI 28nm for intelligent embedded systems," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2017, pp. 238–239.
- [15] B. Moons et al., "Minimum energy quantized neural networks," in *Proc. 51st Asilomar Conf. Signals, Syst., Comput.*, Oct. 2017, pp. 1921–1925.

- [16] Y. Ma et al., “An automatic RTL compiler for high-throughput FPGA implementation of diverse deep convolutional neural networks,” in *Proc. 27th Int. Conf. Field-Programm. Log. Appl. (FPL)*, Sep. 2017, pp. 1–8.
- [17] J. Qiu et al., “Going deeper with embedded FPGA platform for convolutional neural network,” in *Proc. ACM/SIGDA Int. Symp. Field-Programm. Gate Arrays*, Feb. 2016, pp. 26–35.
- [18] Y. Umuroglu et al., “FINN: A framework for fast, scalable binarized neural network inference,” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, Feb. 2017, pp. 65–74.
- [19] Lattice. *Lattice SensAI Delivers 10X Performance Boost for Low-Power, Smart IoT Devices At the Edge*. Accessed: Sep. 2020. [Online]. Available: <https://www.latticesemi.com/About/Newsroom/PressReleases/2019/201911sensAI>
- [20] (2019). *Raspberry Pi Compute Module 3+*. 2019. [Online]. Available: https://www.raspberrypi.org/documentation/hardware/computemodule/datasheets/rpi_DATA_-CM3plus_-1p0.pdf
- [21] (2018). *2018. Kendryte: K210 Datasheet*. [Online]. Available: https://s3.cn-north-1.amazonaws.com.cn/dl.kendryte.com/documents/kendryte?_datasheet?_-2018101163248?_en.pdf
- [22] D. E. J. Yiu. *Introduction to the Arm Cortex-m55 Processor*. Accessed: Feb. 2020. [Online]. Available: <https://pages.arm.com/cortex-m55-introduction.html>
- [23] E. Flamand et al., “GAP-8: A RISC-V SoC for AI at the edge of the IoT,” in *Proc. IEEE 29th Int. Conf. Appl.-Specific Syst., Archit. Processors (ASAP)*, Jul. 2018, pp. 1–4.
- [24] L. Lai, N. Suda, and V. Chandra, “CMSIS-NN: Efficient neural network kernels for arm Cortex-M CPUs,” 2018, *arXiv:1801.06601*.
- [25] M. Rusci et al., “Work-in-progress: Quantized NNs as the definitive solution for inference on low-power ARM MCUs?” in *Proc. Int. Conf. Hardw./Softw. Codesign Syst. Synth. (CODES+ISSS)*, Sep. 2018, pp. 1–2.
- [26] A. Garofalo et al., “XpulpNN: Accelerating quantized neural networks on RISC-V processors through ISA extensions,” in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2020, pp. 186–191.
- [27] A. Burrello et al., “DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs,” *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1253–1268, Aug. 2021.
- [28] S. Höppner et al., “How to achieve world-leading energy efficiency using 22FDX with adaptive body biasing on an arm cortex-M4 IoT SoC,” in *Proc. 49th Eur. Solid-State Device Res. Conf. (ESSDERC)*, Sep. 2019, pp. 66–69.
- [29] *Introduction to the Arm Cortex-m55 Processor*, ARM Limited, Cambridge, U.K., 2020. Accessed: Dec. 8, 2024.
- [30] M. Cavalcante et al., “Spatz: A compact vector processing unit for high-performance and energy-efficient shared-11 clusters,” in *Proc. 41st IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2022, pp. 1–9.

Angelo Garofalo is an assistant professor at the University of Bologna, 40126 Bologna, Italy. He is also a post-doctoral researcher at ETH Zürich, Zürich, Switzerland. His research focuses on heterogeneous computing architectures for Edge AI, energy-efficient multi-core processors, and reliable, time-predictable mixed-criticality systems-on-chip. Garofalo has an MSc in electronic engineering and a PhD from the University of Bologna. He is a Member of IEEE.

Luca Benini is the chair of digital circuits and systems at ETH Zürich, 8092 Zürich, Switzerland. He is also a full professor at the University of Bologna, Bologna, Italy. His research interests include energy-efficient parallel computing systems, smart sensing micro-systems, and machine learning hardware. Benini has a PhD from Stanford University, Stanford, CA, USA. He is a Fellow of IEEE and ACM.

■ Direct questions and comments about this article to Angelo Garofalo, University of Bologna, 40126 Bologna, Italy; angelo.garofalo@unibo.it.